



Universidad de Buenos Aires
Facultad de Ingeniería
Departamento de Electrónica

Taller de Automatización y Control (TA135)

Trabajo Práctico 1

Introducción al control digital

Integrantes:

Apellido y Nombre	Padrón	Email
Romero, Brandon	105004	boromero@fi.uba.ar
Rozenberg, Francisco Samy	109836	frozenberg@fi.uba.ar

Segundo Cuatrimestre 2026

Índice

1. Introducción	2
2. Limitaciones de los sensores	2
2.1. Potenciómetro	2
2.1.1. Resolución en tensión	3
2.1.2. Resolución en grados	3
2.1.3. Tiempo de medición	4
2.2. Sensor de distancia	4
2.2.1. Resolución temporal y de distancia	4
2.2.2. Tiempo de medición	5
2.3. IMU	7
2.3.1. Resolución angular	9
2.3.2. Tiempo de medición	9
3. Limitaciones de los actuadores	10
3.1. Servomotor utilizado	10
3.2. Resolución angular del servomotor	11
3.3. Tiempo para mover 30 grados	12
3.4. Control a 1 Hz	15
4. Transmisión por puerto serie	15
4.1. Experimento realizado	15

1. Introducción

A lo largo de la asignatura se estudia la implementación de un sistema de control digital. El objetivo del mismo es controlar la posición de un carrito sobre una barra mediante la modificación de su inclinación. Para ello se utiliza un servomotor como actuador y diferentes sensores para obtener información sobre la posición y orientación del sistema.

El objetivo de este informe es analizar las limitaciones de los distintos sensores y actuadores utilizados en la planta, considerando tanto sus características físicas como las restricciones impuestas por la implementación sobre Arduino.

Se estudia un potenciómetro lineal, un sensor de distancia ultrasónico, una unidad de medición inercial (IMU por sus siglas en inglés) y un servomotor. A la vez se analiza el tiempo requerido para realizar mediciones y transmitir información mediante el puerto serie.

2. Limitaciones de los sensores

Para caracterizar y en un futuro controlar la planta de manera fiable, es necesario conocer las características y limitaciones físicas de los distintos sensores y actuadores presentes en el sistema.

2.1. Potenciómetro

Se utiliza un potenciómetro estándar *B10K* de 3/4 de vuelta para fijar un ángulo de referencia. Esta función se lleva a cabo mediante el comando `analogRead()` de la librería estándar de Arduino, que mapea valores analógicos entre 0V y 5V a números enteros entre el 0 y 1023 mediante el convertidor analógico-digital de 10 bits interno de la placa. Las mediciones experimentales referenciadas en esta sección se realizan mediante el script Lst. 1

```
1  #define PIN_POTE A1
2
3  void setup() {
4
5      Serial.begin(115200);
6      pinMode(PIN_POTE, "input");
7
8  }
9
10 void loop() {
11
12     unsigned long t_inicial = micros();
13     float lectura = analogRead(A0);
14     unsigned long t_final = micros();
15
16     Serial.print("Lectura:");
17     Serial.print(lectura*5/1023,3);
18     Serial.print("V_/");
19     Serial.print(lectura*270/1023,3);
20     Serial.print("deg");
21     Serial.print("Tiempo:");
22     Serial.print(t_final - t_inicial);
23     Serial.println("us");
24 }
```

Listado 1: Script de medición del potenciómetro

El cual imprime una serie de resultados con las características de Cua. 1

Lectura [V]	Lectura [°]	Tiempo de medición [μs]
0.489	26.393	116
0.484	26.129	108
0.489	26.393	112
0.484	26.129	112

Cuadro 1: Lecturas del Potenciómetro

2.1.1. Resolución en tensión

Dividiendo el intervalo de tensión por la cantidad de escalones disponibles en el ADC:

$$\frac{5V - 0V}{1023} = 4,88mV. \quad (1)$$

Sin embargo, realizando una medición experimental de este valor se determina que, debido a las características del sistema utilizado, solamente es posible obtener mediciones certeras con una resolución de $10mV$.

2.1.2. Resolución en grados

El potenciómetro utilizado en este proyecto posee un recorrido de 0° a 270° . Realizando un procedimiento similar al de la sección anterior se llega a la resolución máxima:

$$\frac{270^\circ - 0^\circ}{1023} \approx 0,264^\circ. \quad (2)$$

A diferencia de la medición anterior, este umbral resulta adecuado para obtener mediciones fiables del ángulo de rotación del dispositivo.

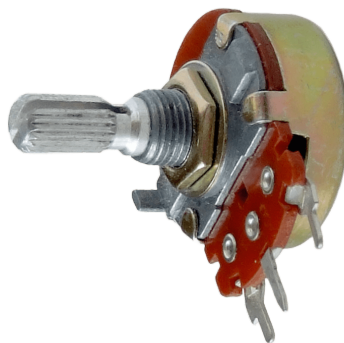


Figura 1: Potenciómetro lineal utilizado

2.1.3. Tiempo de medición

Observando la especificación de la función `analogRead()` se determina que al Arduino le toma 1775 ciclos de *clock* ejecutar esta función. Luego, teniendo en cuenta la frecuencia de reloj del procesador ATMEGA328P, que es de 16MHz , se define que el tiempo nominal que se tarda en leer una muestra del potenciómetro es de $111\mu\text{s}$. El tiempo que se mide mediante Lst. 1 se encuentra en un margen de error de este valor dando un resultado de $116\mu\text{s}$.

2.2. Sensor de distancia

Para la medición de distancia se utiliza un sensor ultrasónico HC-SR04. Este módulo permite determinar la distancia a un objeto mediante la emisión y recepción de ondas ultrasónicas. El sensor posee cuatro terminales: VCC, GND, TRIG y ECHO. Los terminales VCC y GND se utilizan para alimentar el módulo, mientras que TRIG y ECHO permiten iniciar una medición y obtener el tiempo de vuelo de la señal de ultrasonido.

Para iniciar una medición, se aplica un pulso de aproximadamente $10\mu\text{s}$ en el terminal TRIG. Luego, el HC-SR04 emite una ráfaga de ocho ciclos de ultrasonido a una frecuencia de 40kHz . Una vez emitida la señal, el terminal ECHO pasa a nivel alto y permanece en este estado hasta que el sensor recibe el eco producido por la reflexión de la onda en un objeto. La duración del pulso presente en ECHO permite determinar el tiempo que tardó la onda en realizar el recorrido de ida y vuelta.

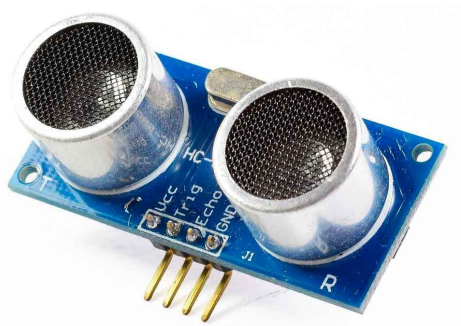


Figura 2: Sensor ultrasónico HC-SR04 utilizado.

2.2.1. Resolución temporal y de distancia

El tiempo durante el cual ECHO permanece en nivel alto corresponde al tiempo que tarda la onda en realizar el recorrido de ida y vuelta. Por lo tanto, la distancia se obtiene a partir :

$$d = \frac{v_{\text{sonido}} t}{2}$$

donde v_{sonido} representa la velocidad de propagación del sonido en el aire y t es el tiempo medido mediante el terminal ECHO. El factor $1/2$ se debe a que el tiempo medido corresponde tanto al recorrido de ida como al de vuelta.

Considerando una velocidad del sonido de aproximadamente 343 m/s , una variación temporal Δt se relaciona con una variación de distancia Δd mediante

$$\Delta d = \frac{v_{\text{sonido}} \Delta t}{2}$$



Según las especificaciones del fabricante, el sensor presenta una resolución mínima en distancia de aproximadamente 0,3 cm. La resolución temporal equivalente puede obtenerse despejando Δt de la expresión anterior:

$$\Delta t = \frac{2\Delta d}{v_{\text{sonido}}}$$

Por lo tanto,

$$\Delta t = \frac{2(0,003 \text{ m})}{343 \text{ m/s}} \approx 17,5 \mu\text{s}.$$

Es decir, una resolución de 0,3 cm corresponde aproximadamente a una resolución temporal equivalente de 17,5 μs en el tiempo de vuelo de la onda ultrasónica.

2.2.2. Tiempo de medición

Para determinar el tiempo necesario para realizar una medición con el sensor HC-SR04, se utilizó la función `sonar.ping()` de la biblioteca NewPing. Se realizó el siguiente código, que permite comparar el tiempo total empleado por la función con el tiempo de vuelo de la onda del ultrasonido.

```
1 void loop() {
2   // Comparo el tiempo de vuelo de la medicion con el tiempo
3   // de procesamiento del dato
4
5   unsigned long ti_med = micros();
6   unsigned int medicion_cruda = sonar.ping(MAX_DISTANCE);
7   unsigned long tf_med = micros();
8
9   Serial.print("Tiempo_medicion:_");
10  Serial.print((tf_med - ti_med));
11  Serial.println("_us");
12
13  Serial.print("Distancia:_");
14  Serial.print(medicion_cruda);
15  Serial.println("_us");
16
17  Serial.println((tf_med - ti_med) - medicion_cruda);
18  delay(1000);
19 }
```

Listado 2: Script de medición del sensor de distancia

Al comienzo de la medición se almacena el tiempo mediante la función `micros()`, obteniéndose $t_{i,med}$. Luego se ejecuta `sonar.ping(MAX_DISTANCE)`, que inicia la medición y devuelve el tiempo de vuelo de la señal en microsegundos. Finalmente, se vuelve a utilizar `micros()` para obtener $t_{f,med}$.

Por lo tanto, la diferencia

$$t_{f,med} - t_{i,med}$$

representa el tiempo total empleado para realizar la medición mediante la función `sonar.ping()`. Este tiempo está compuesto por el tiempo de vuelo de la onda más el tiempo necesario para el procesamiento de la medición:

$$T_{medicion} = T_{vuelo} + T_{procesamiento}$$

Para determinar el tiempo de procesamiento, se restó al tiempo total de medición el tiempo de vuelo devuelto por `sonar.ping()`:

$$T_{procesamiento} = T_{medicion} - T_{vuelo}$$

Se realizaron varias mediciones, obteniéndose los siguientes resultados:

Tiempo de medición [μs]	Tiempo de vuelo [μs]	Procesamiento [μs]
656	419	237
656	419	237
656	423	233
652	423	233
656	419	237
656	423	233
680	443	237
652	423	237
656	419	237
656	423	233

Cuadro 2: Mediciones del tiempo de ejecución de `sonar.ping()`.

A partir de las mediciones se observa que el tiempo de procesamiento presenta pequeñas variaciones, con valores de 233 y 237 μs , siendo 237 μs el valor mas frecuente y la cota máxima.

Tomando como referencia el valor de 237 μs , el tiempo necesario para realizar una medición puede expresarse aproximadamente como

$$T_{medicion} = T_{vuelo} + 237 \mu s$$

De esta forma, se observa que el tiempo requerido para realizar una medición no está determinado únicamente por el recorrido de la onda de ultrasonido, sino que también existe un tiempo adicional asociado al procesamiento de la medición.

Finalmente, se comparó el tiempo necesario para realizar una medición con el período de muestreo utilizado en el sistema de control. Para una frecuencia de muestreo de 50 Hz, el período disponible es

$$T_{muestreo} = \frac{1}{50 \text{ Hz}} = 20 \text{ ms.}$$

La máxima distancia que se desea medir en el sistema es de 35 cm. Considerando una velocidad del sonido de aproximadamente 343 m/s, el tiempo de vuelo correspondiente a esta distancia es

$$T_{vuelo} = \frac{2d}{v_{sonido}} = \frac{2(0,35 \text{ m})}{343 \text{ m/s}} \approx 2,04 \text{ ms.}$$

A este tiempo se debe sumar el tiempo de procesamiento de la medición, que experimentalmente se consideró aproximadamente igual a $237 \mu s$. Por lo tanto, el tiempo total necesario para realizar una medición a la máxima distancia es

$$T_{\text{medición}} = T_{\text{vuelo}} + T_{\text{procesamiento}}$$

$$T_{\text{medición}} = 2,04 \text{ ms} + 0,237 \text{ ms} \approx 2,277 \text{ ms}.$$

Comparando este resultado con el período de muestreo de 20 ms, se observa que

$$2,277 \text{ ms} < 20 \text{ ms}.$$

2.3. IMU

Este dispositivo combina un acelerómetro y un giroscopio de tres ejes con *ADC* internos de 16 bits. Para estimar la orientación angular en torno al eje de rotación de la barra de forma robusta, se busca aminorar los efectos del ruido y del corrimiento a lo largo del tiempo propio de las técnicas de integración usadas para obtener el dato. Esto se realiza mediante el filtro complementario implementado en Lst. 3 y Lst. 4.

Al encender el sistema, se realiza una calibración inicial en reposo promediando 500 muestras de velocidad angular en el eje $\hat{X}$, obteniendo el sesgo estático del giroscopio según (3). Este proceso se explicita en Lst. 3:

$$sesgo_X = \frac{1}{500} \sum_{k=1}^{500} \omega_{x,k}. \quad (3)$$

Durante el bucle de ejecución, el acelerómetro genera su propia estimación del ángulo de rotación a partir de las componentes a_y y a_z . Esto se realiza a partir de la medición de las componentes del vector de aceleración gravitatoria:

$$\alpha_a = \text{atan2}(a_y, a_z) \quad (4)$$

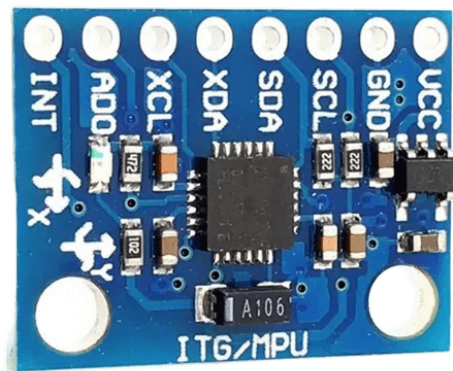


Figura 3: Potenciómetro lineal utilizado



```
1 void setup() {
2
3   Serial.begin(115200);
4   while (!Serial) {
5     delay(10);
6   }
7
8   if (!mpu.begin()) {
9     Serial.println("NO_SE_ENCONTRO_EL_MPU6050");
10    while (1) {
11      delay(10);
12    }
13  }
14  mpu.setAccelerometerRange(MPU6050_RANGE_8_G);
15  mpu.setGyroRange(MPU6050_RANGE_500_DEG);
16  mpu.setFilterBandwidth(MPU6050_BAND_44_HZ);
17  float suma = 0;
18  int ciclos = 500;
19  for (int t = 0; t < ciclos; t++) {
20    sensors_event_t a, g, temp;
21    mpu.getEvent(&a, &g, &temp);
22    suma = suma + g.gyro.x;
23    delay(10);
24  }
25  sesgo_x = suma / ciclos;
26  [...]
27  delay(1000);
28 }
```

Listado 3: Configuración para muestreo con MPU 6050

A la vez, se integran las mediciones del giroscopio, compensando la deriva previamente nombrada:

$$\alpha_g(k) = \alpha(k-1) + (\omega_x(k) - \omega_{off}) \cdot T_s \quad (5)$$

La estimación se produce mediante la expresión del filtro complementario (6):

$$\alpha(k) = R \cdot \alpha_g(k) + (1 - R) \cdot \alpha_a(k) \quad (6)$$

donde se fija $R = 0,85$, asignando una ponderación del 85 % a la respuesta del giroscopio y un 15 % a la referencia del acelerómetro.

```
1 void loop() {
2
3   const int muestras = 200;
4   float media = 0;
5   float tiempo_total = 0;
6   float M2 = 0;
7   static float alpha = 0;
8   for (int i = 1; i <= muestras; i++) {
9     unsigned long t_inicial = micros();
10    sensors_event_t a, g, temp;
11    mpu.getEvent(&a, &g, &temp);
12    float alpha_a = atan2(a.acceleration.y, a.acceleration.z);
13    alpha = alpha + (g.gyro.x - sesgo_x) * (T_MUESTREO * 1e-6);
14    float R = 0.85;
15    alpha = alpha * R + alpha_a * (1 - R);
16    float delta = alpha - media;
```



```
17     media = media + delta / i;
18     M2 = M2 + delta * (alpha - media);
19     unsigned long t_final = micros();
20     unsigned long tiempo_medicion = t_final - t_inicial;
21     tiempo_total = tiempo_total + tiempo_medicion;
22
23     [...]
24     delay(20);
25 }
26
27 float varianza = M2 / muestras;
28 float desvio = sqrt(varianza);
29 float tiempo_promedio = tiempo_total / muestras;
30
31 [...]
32 }
```

Listado 4: Adquisición y filtrado complementario con MPU 6050

2.3.1. Resolución angular

La resolución angular y la estadística de la toma de muestras se analizan procesando un lote de $N = 200$ muestras en reposo de manera recursiva. El cálculo de la media $\bar{\mathbb{E}}$ y la varianza σ^2 se define en (7):

$$\bar{\mathbb{E}} = \frac{1}{N} \sum_{i=1}^N \mathbb{E}_i, \quad \sigma^2 = \frac{1}{N} \sum_{i=1}^N (\mathbb{E}_i - \bar{\mathbb{E}})^2 \quad (7)$$

El fabricante del MPU 6050 especifica para el acelerómetro un piso de ruido de $400 \mu g / \sqrt{\text{Hz}}$ y para el giróscopo de $0,005^\circ / \text{s} / \sqrt{\text{Hz}}$. Configurando el filtro pasa-bajos digital interno en 44Hz (MPU6050_BAND_44_HZ), el sistema atenúa el ruido de alta frecuencia. En las lecturas experimentales tabuladas en Cua. 3, se obtiene una desviación estándar de $\sigma = 0,0185^\circ$ y una varianza de $\sigma^2 = 0,000343$ frente a un ángulo medio de $-20,3814^\circ$.

2.3.2. Tiempo de medición

Como se aprecia en Cua. 3, el tiempo medio de adquisición resulta $\bar{t}_{med} \approx 3,38 \text{ ms}$. Este valor responde al tiempo que requiere la comunicación I2C. En comparación, la hoja de datos del fabricante indica tiempos de medición en modo estándar de hasta $1,53 \text{ ms}$. La amplia diferencia entre valores se explica por el *overhead* que imponen la librería usada y la arquitectura de Arduino sobre el protocolo de comunicación.

Muestra	Lectura [°]	Tiempo de medición [μs]
1	-20,3967	3364
2	-20,3670	3380
3	-20,3530	3392
4	-20,3663	3392
5	-20,3671	3392
6	-20,3684	3388
7	-20,3638	3380
8	-20,3631	3384
9	-20,3524	3376
10	-20,3605	3380
Promedio ($N = 200$)	-20,3814	3383,44
Desvío estándar (σ)	0,0185	6,52

Cuadro 3: Lecturas del MPU 6050 y tiempos de medición I2C

3. Limitaciones de los actuadores

3.1. Servomotor utilizado

Para el accionamiento de la barra se utiliza un servomotor MG996R. Este servomotor permite controlar su posición angular mediante una señal de control PWM.



Figura 4: Servomotor MG996R.

Las principales características indicadas por el fabricante para el servomotor utilizado son:



Weight [g]	55
Dimension [mm]	40.7 × 19.7 × 42.9
Stall Torque [kgf · cm]	9.4 (4.8 V), 11 kgf (6 V)
Operating speed [s/60°]	0.17 (4.8 V), 0.14 (6 V)
Operating voltage [V]	4.8 – 7.2
Running current [mA]	500 – 900(6 V)
Stall current [A]	2.5 (6 V)
Dead band width [μs]	5
Temperature range [°C]	0 – 55

3.2. Resolución angular del servomotor

Para determinar experimentalmente la mínima variación del ancho de pulso capaz de producir un movimiento apreciable del servomotor, se utiliza la función `writeMicroseconds()`. Se toma como posición inicial un ancho de pulso de $1500 \mu s$ y se incrementa progresivamente este valor hasta observar un movimiento del servomotor.

El ensayo se realiza mediante el siguiente código:

```

1 void loop() {
2
3 unsigned long t_inicial = micros();
4
5 static int T_BARRA = 1500; // posicion central
6 static int contador = 0;
7 contador++;
8
9 if (contador == 50) {
10     contador = 0;
11     if (T_BARRA == 1500) {
12         T_BARRA = 1520;
13     } else {T_BARRA = 1500;}
14 }
15 servo.writeMicroseconds(T_BARRA);
16 unsigned long t_final = micros();
17 delay((T_MUESTREO - (t_final - t_inicial)) / 1000);
18 }

```

Listado 5: Script de medición del servomotor

El período de muestreo utilizado es de 20ms. El contador permite mantener cada posición durante 50 ciclos, es decir:

$$50 \cdot 20 \text{ ms} = 1000 \text{ ms}$$

Por lo tanto, el servomotor permanece aproximadamente 1 s en $1500 \mu s$ y luego 1 s en $1520 \mu s$, permitiendo observar si el cambio produce un movimiento. Mediante este procedimiento se observa que el menor diferencial que produce un movimiento apreciable en el servo es de :

$$\Delta T_{\text{exp}} = 1520 \mu s - 1500 \mu s$$



$$\Delta T_{\text{exp}} = 20 \mu s$$

Para convertir esta variación temporal en una variación angular, se considera el rango de funcionamiento del servomotor de 180° , asociado aproximadamente a un rango de $2000 \mu s$. De esta manera:

$$\frac{180^\circ}{2000 \mu s} = 0,09^\circ / \mu s$$

Por lo tanto, el diferencial observado experimentalmente corresponde a:

$$\Delta \theta_{\text{servo,exp}} = 20 \mu s \frac{180^\circ}{2000 \mu s}$$

$$\Delta \theta_{\text{servo,exp}} \approx 1,8^\circ$$

Si se considera únicamente la resolución teórica del ancho de pulso, un cambio de $1 \mu s$ permitiría comandar una variación angular de:

$$\Delta \theta_{\text{servo,teor}} = 1 \mu s \frac{180^\circ}{2000 \mu s}$$

$$\Delta \theta_{\text{servo,teor}} \approx 0,09^\circ$$

Sin embargo, el servomotor presenta un *Dead Band Width* de $5 \mu s$. Esto significa que variaciones del ancho de pulso menores a este valor pueden no producir un movimiento apreciable del servomotor. Considerando este valor, la variación angular asociada es:

$$\Delta \theta_{\text{DeadBand}} = 5 \mu s \frac{180^\circ}{2000 \mu s}$$

$$\Delta \theta_{\text{DeadBand}} \approx 0,45^\circ$$

Por lo tanto, se diferencia entre la resolución teórica del comando, de aproximadamente $0,09^\circ$, la resolución asociada al *Dead Band*, de aproximadamente $0,45^\circ$, y la mínima variación observable experimentalmente, de aproximadamente $1,8^\circ$. La diferencia entre estos valores se debe a que la resolución temporal del comando y el *Dead Band* no determinan por sí solos la capacidad de movimiento del servomotor. En la práctica también intervienen factores como la fricción con el ruleman, la electrónica interna de este servo en específico, la carga mecánica de la barra y las características del montaje.

3.3. Tiempo para mover 30 grados

Para determinar experimentalmente el tiempo necesario para que la barra se desplace 30° , se utilizó el acelerómetro IMU MPU6050 para estimar el ángulo de la barra. El ángulo se calculó a partir de las componentes y y z de la aceleración mediante:



$$\theta = \text{atan2}(a_y, a_z) \frac{180^\circ}{\pi}$$

El procedimiento consistió en llevar inicialmente el servomotor a la posición correspondiente a $2200 \mu s$, esperar $1 s$ para que el sistema se estabilizara y medir el ángulo inicial de la barra. Luego se comandó la posición correspondiente a $1100 \mu s$ y se midió el tiempo transcurrido hasta que la diferencia entre el ángulo inicial y el ángulo medido superara los 30° .

El código utilizado para realizar la medición fue:

```
1 void loop() {
2
3   // diferencial de 30 grados
4
5   servo.writeMicroseconds(POSICION_INICIAL);
6   delay(1000);
7   sensors_event_t a, g, temp;
8   mpu.getEvent(&a, &g, &temp);
9   float angulo_inicial = atan2(a.acceleration.y, a.acceleration.z) * 180.0 / PI;
10  int angulo_servo = servo.read();
11
12  Serial.print("Angulo_inicial_barra:");
13  Serial.print(angulo_inicial, 3);
14  Serial.println("_grados");
15
16  Serial.print("Angulo_servo:");
17  Serial.print(angulo_servo, 3);
18  Serial.println("_grados");
19
20  for (int i = 0; i < 10000; i++) {
21    if (i == 0) {
22      t_inicial = millis();
23      servo.writeMicroseconds(POSICION_FINAL);
24    } else {
25      mpu.getEvent(&a, &g, &temp);
26      float angulo_final = atan2(a.acceleration.y, a.acceleration.z) * 180.0 / PI;
27      if (abs(angulo_final - angulo_inicial) > 30) {
28        unsigned long t_final = millis();
29        int angulo_servo_f = servo.read();
30
31        Serial.print("Angulo_final:");
32        Serial.print(angulo_final, 3);
33        Serial.println("_grados");
34        Serial.print("Tiempo_para_mover_30_grados:");
35        Serial.print(t_final - t_inicial - 1); // RESTO 1 SON LOS DELAYS
36        Serial.println("_ms");
37        Serial.print("Movimiento_del_servo:");
38        Serial.print(angulo_servo_f - angulo_servo);
39        Serial.println("grados");
40        Serial.print("Relacion_entre_movimiento_de_barra_y_servo:");
41        Serial.println((angulo_servo_f - angulo_servo) / (angulo_final - angulo_inicial));
42        ;
43        break;
44      }
45    }
46    delay(1);
47  }
48  delay(10000);
49 }
```

Listado 6: Script de medición del servomotor

Se realizaron seis mediciones, obteniéndose los siguientes tiempos:

Ensayo	$\theta_i (^{\circ})$	$\theta_f (^{\circ})$	$t (ms)$
1	-20,435	11,194	404
2	-20,391	9,903	402
3	-20,464	9,545	400
4	-20,646	10,787	403
5	-20,441	10,517	424
6	-20,513	9,519	395

Los tiempos medidos presentan una dispersión debido principalmente a las características del movimiento mecánico y a la medición realizada mediante la IMU. El valor medio obtenido fue:

$$\bar{t}_{exp} = \frac{404 + 402 + 400 + 403 + 424 + 395}{6}$$

$$\bar{t}_{exp} \approx 404,7 ms$$

Por lo tanto, experimentalmente se obtuvo un tiempo aproximado de:

$$t_{30^{\circ}} \approx 405 ms$$

Durante las mediciones también se observó que el cambio de posición del servomotor fue aproximadamente de 106° , mientras que el desplazamiento correspondiente de la barra fue de aproximadamente 30° . La relación entre ambos movimientos resultó aproximadamente:

$$\left| \frac{\Delta\theta_{servo}}{\Delta\theta_{barra}} \right| \approx 3,4$$

El datasheet del servomotor MG996R especifica una velocidad de funcionamiento de:

$$0,17 s/60^{\circ}$$

Considerando que en el experimento el servomotor se desplazó aproximadamente 106° , el tiempo esperado según esta especificación sería:

$$t_{datasheet} = 0,17 s \frac{106^{\circ}}{60^{\circ}}$$

$$t_{datasheet} \approx 0,300 s$$



o equivalentemente:

$$t_{datasheet} \approx 300 \text{ ms}$$

El tiempo experimental obtenido, aproximadamente 405 ms , resulta mayor que el valor estimado a partir de la velocidad especificada por el fabricante. La diferencia puede deberse a que la especificación del datasheet corresponde a condiciones determinadas de funcionamiento, mientras que la medición se realizó sobre el sistema completo, incluyendo la carga mecánica de la barra, la transmisión de movimiento entre el servo y la barra y las características dinámicas del montaje.

Además, el tiempo experimental se determinó utilizando la medición del ángulo mediante el acelerómetro del MPU6050 y un umbral de 30° , por lo que también intervienen la frecuencia de medición, el tiempo de ejecución del programa y la resolución de la medición angular.

3.4. Control a 1 Hz

En los ensayos anteriores se utilizó una frecuencia de control de 50 Hz , correspondiente a un período de muestreo de 20 ms . Si la frecuencia de control se redujera a 1 Hz , el período entre actualizaciones de la referencia sería de 1 segundo . La resolución angular obtenida en el primer ensayo no depende de la frecuencia de actualización, sino de la resolución con la que se puede modificar el ancho del pulso. Por lo tanto, la resolución teórica del comando seguiría siendo la misma. Por otro lado, la velocidad mecánica del servomotor tampoco se modifica directamente al cambiar la frecuencia de actualización del controlador. Por lo tanto, el tiempo experimental obtenido para mover la barra 30° es exactamente el mismo.

La principal diferencia se produce en la frecuencia con la que el controlador puede modificar la referencia del servomotor. A 50 Hz , una nueva orden puede ser enviada cada 20 ms , mientras que a 1 Hz solamente se puede actualizar cada 1 s . Por lo tanto, al utilizar una frecuencia de control de 1 Hz , el sistema tendría una respuesta de control mucho más lenta.

4. Transmisión por puerto serie

4.1. Experimento realizado

Para estimar el tiempo necesario para transmitir 100 valores de tipo float por el puerto serie, se configuró la comunicación del Arduino a una velocidad de 115200 bps . Cada float ocupa 4 bytes, por lo que el envío de 100 valores representa un total de 400 bytes

Se utilizó el siguiente código, en el cual se mide mediante las funciones `micros()`:

```
1 void setup() {
2   Serial.begin(115200);
3 }
4
5 void loop() {
6
7   float dato = 1.2;
8   unsigned long t_inicial = micros();
9
10  for (int i = 0; i < 100; i++) {
11    Serial.write((byte*)&dato, 4);
12  }
```



```
13 Serial.flush();
14 unsigned long t_final = micros();
15 Serial.println();
16 Serial.print("Tiempo_de_transmision:_");
17 Serial.print(t_final - t_inicial);
18 Serial.println("_us");
19
20 delay(1000);
21 }
```

Listado 7: Medición del tiempo de transmisión de 100 floats

El experimento se repitió varias veces, obteniéndose valores entre $34496 \mu s$ y $35500 \mu s$. El tiempo promedio medido fue aproximadamente:

$$t_{\text{medido}} \approx 35,1 \text{ ms}$$

Para estimar el tiempo real de transmisión se debe considerar que, en una comunicación serie UART convencional, cada byte requiere 10 bits: 1 bit de inicio, 8 bits de datos y 1 bit de parada. Por lo tanto, para 400 bytes se necesitan:

$$400 \text{ bytes} \times 10 \frac{\text{bits}}{\text{byte}} = 4000 \text{ bits}$$

A una velocidad de 115200 bps, el tiempo de transmisión es:

$$t = \frac{4000}{115200}$$

$$t \approx 34,72 \text{ ms}$$

No resulta conveniente medir directamente el tiempo de transmisión de un único `float`, ya que el tiempo involucrado es muy pequeño y la medición queda afectada por el funcionamiento del buffer de transmisión y por la resolución y el tiempo de ejecución de las funciones utilizadas para medirlo. Por este motivo, se transmiten 100 `float` y luego se estima el tiempo correspondiente a un único valor.